Goto Dependency Definition for Haskell Language Server

Google Summer of Code 2026

March 2026

Problem Statement

Haskell Language Server (HLS) currently supports *Goto Definition* only for functions defined within a user's project. This project extends the feature to external dependencies, allowing developers to navigate directly to the implementation of library functions from their IDE. By indexing `.hie` files generated by GHC using tools such as `hiedb`, HLS can locate and open source definitions from third-party libraries.

Student: Vidit Odedra

Organization: Haskell.org

Project: Haskell Language Server

Mentors: Fendor, wz1000

1 About Me

The Student

Name Vidit Odedra

e-mail vidit894@gmail.com

Github https://github.com/vidit-od

The Institute

University National Institute of Technology, Tiruchirappalli

Major Computer Science and Engineering

Year Second Year

Degree Master of Technology (M.Tech)

Background & Programming Experience

I have been programming for approximately four years. I began with C and C++, focusing on building a strong foundation in programming fundamentals and data structures and algorithms while developing several small projects. Over time, I expanded into domain-specific technologies, learning JavaScript for web development and Python for machine learning. During this period, I worked on multiple projects, completed two internships, and co-authored two research publications related to these areas.

Around one and a half years ago, I started working with TypeScript and developed a strong interest in statically typed languages. This interest led me to explore Haskell, where I became particularly interested in its type system and functional programming paradigm. Over the past few months, I have been actively learning Haskell and contributing to the Haskell Language Server (HLS), gaining practical experience with the codebase

Contributions

I started contributing to the Haskell Language Server (HLS) in January 2026. Since then, I have been actively contributing to the project while learning from the community and gaining familiarity with the HLS codebase. Below is a summary of my contributions and involvement with the project.

Merged PR's

The following pull requests represent my **direct contributions** that were reviewed and **merged** into the Haskell Language Server master branch.

- 4791 – *Improving Notes Plugin*
- 4813 – *Cursor aware record expansion*
- 4823 – *Smart Casing in modules and file paths*

- 4828 – *Fix notes crashing and leaking single comments notes*
- 4841 – *Fix spacing issues while refactor-reduce*
- 4848 – *Custom-setups completion in cabal files*
- 4856 – *Use Server side Diagnostics*
- 4857 – *Produce structured error for unknown module*
- 4696 - *Upgrade Stack Snapshots*
- 4865 – *Migrate stackage build on GHA*
- 4868 – *Fix cache cleaning job*

Open PRs

The following pull requests represent my **direct contributions** that are currently **under review** and have not yet been merged into the Haskell Language Server master branch.

- 4845 – *Cross Module renaming*
- 4849 – *Migrate HLS-refactor-plugin to use Structured Diagnostic*

Other Contributions

This section highlights **indirect contributions** where I investigated issues and reported bugs that were later resolved by other HLS contributors.

- 4843 - *Double hashes in ghcide cache path causes long path*
- 4834 - *Use VirtualFileTree in hls-call-hierarchy-plugin-tests*
- 4863 - *Pin version of extra to 1.8.1 for stack*
- 4870 - *Discussion on HLS cache management.*

2 The Project

2.1 Objective

What : Goto dependency is a feature provided by language servers that improves the user's context on imported symbols. Triggering this on a symbol allows users to view the signature and the underlying implementation of a function they are importing. This helps them understand the abstracted components used within a code block.

Why : In larger codebases, developers often work with many abstracted symbols that originate from third-party libraries. The current workflow typically involves searching for the corresponding library, locating the symbol manually, and then accessing its implementation. This interrupts the development flow and requires developers to leave their IDE.

Limitation : Currently, haskell-language-server provides goto definition functionality for local projects, i.e., when the imported symbols are present within the project folder. However, for symbols provided by third-party libraries, or symbols whose actual source code does not exist within the project workspace, there is currently no support for this feature.

Goal : The objective of this project is to extend the existing goto definition functionality to support third-party dependencies. This would allow developers to navigate directly to the implementation of imported symbols from external libraries within their IDE, without needing to manually search for the source code online.

2.2 Existing Work

This project was previously attempted in PR #3749. Much of the structural groundwork has already been established there, and this implementation will serve as the base for this project, with further improvements built on top of it.This section outlines the approach taken in that PR.

HIE : When GHC compiles Haskell projects it uses the source files and generates .hi file and object files. Optionally HIE (.HI extended) files are also generated. These contain rich metadata like full typed syntax tree, name bindings and references, source locations and most importantly original source code of the module embedded as a ByteString. These files are used to implement goto dependency defination.

Setup : HIE files are not generated by default during compilation, passing '-fwrite-ide-info' flag during compilation generates them and stores the resulting files inside cabal package store at a predictable location. extraCompilationArtifacts constant inside cabal's source governs this predictable location. using the above flag and constant makes sure that HIE files are always generated and ready to be used by HLS.

HieDb : HLS maintains a SQLite database called HieDb that acts as an index over compiled Haskell modules. It stores metadata extracted from HIE files with their source locations, and paths back to the original source files. HLS uses this database as a fast lookup mechanism when resolving symbol locations, avoiding the need to parse raw HIE files on every query.

By default, HieDb only indexes modules belonging to your own project, dependency modules from third-party libraries have no entries at all. This project extends HieDb's scope by introducing a startup indexing step that discovers HIE files for all transitive dependencies in the Cabal store and registers them in HieDb as placeholder entries, with the source path left empty. We then intercept the point where HLS falls back to HieDb for symbol resolution. At that moment, the corresponding source code is extracted from the HIE file and written to disk, the placeholder entry is upgraded to a real one, and the editor is pointed at the newly written file. This makes it possible to implement goto definition for dependency symbols without requiring their source code to exist on disk ahead of time.

Mechanism : We can implements goto dependency definition through a pipeline of four steps :

1. During HLS startup, a new background task called indexDependencyHieFiles performs a breadth-first traversal of the transitive dependency graph, discovers

all HIE files in the Cabal store for the current project's dependencies, and registers them in HieDb with a FakeFile marker — indicating the HIE file is known but no source file has been written yet.

2. when the user triggers goto definition on a dependency symbol, HLS queries HieDb and finds the module's HIE file path.

3. A new function called lookupMod reads that HIE file from disk, extracts the hie_hs_src field containing the original source code, writes it to a deterministic path inside the project root, upgrades the HieDb row from FakeFile to RealFile, and returns the URI of the newly written file to the editor.

4. when the editor opens this file, HLS can detects that it is a dependency file and serves hover information and further goto definition requests directly from HieDb, bypassing the normal compilation pipeline entirely.

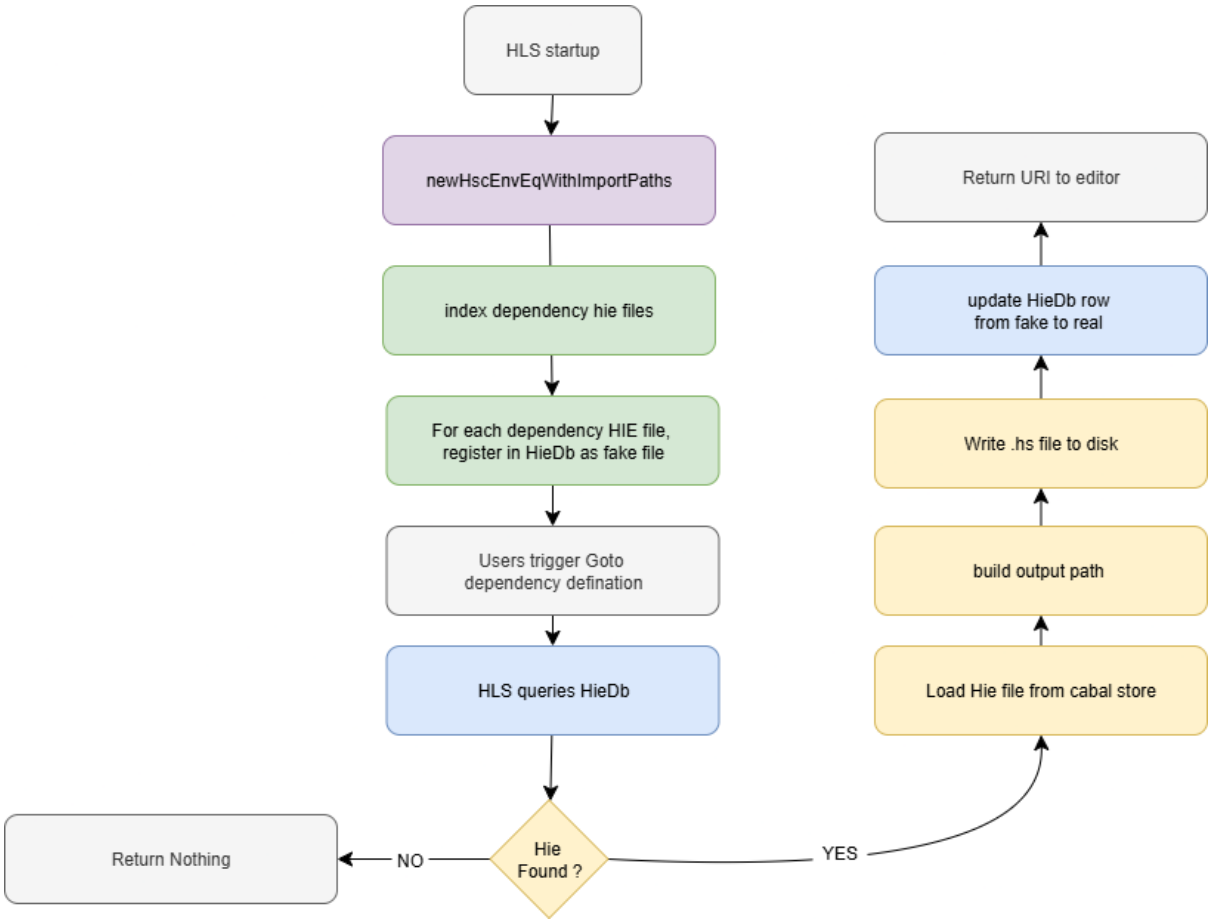

Figure 1: A Flowchart of events to track Goto Dependency defination

Limitations: Despite implementing the core feature, the project remains blocked due to fundamental reliability concerns identified during review. The most significant issue relates to type checking and the triggering of rules on dependency files. When a symbol is opened through goto dependency definition, HLS treats the generated file like any other source file and begins applying its usual rules.

This becomes problematic because these dependency files do not actually exist on disk and therefore lack accompanying project context such as the corresponding '.cabal' files. As a result, the rule system begins to break, leading to a large number of incorrect errors and warnings being presented to the user.

PR #3749 attempted to address the type checking and rule triggering issues by creating a whitelist of rules at runtime. However, this leaves a large number of rules that need to be tracked and maintained to ensure that no unintended rule slips through and causes problems later on. As a result, this approach is fragile and difficult to maintain.

Unlike other third-party libraries that are built by Cabal, additional challenges arise when dealing with boot libraries. As discussed earlier, the '-fwrite-ide-info' flag must be passed during compilation in order to generate HIE files. However, boot libraries are already compiled and distributed with GHC as binary artifacts. GHC cannot be recompiled locally, doing so would take several hours and produce different internal name identifiers, making the resulting HIE files incompatible with the existing binaries. Hence, a way to efficiently deliver boot-lib hie files also needs to be explored.

2.3 Proposed Approach

A significant portion of the foundational work for this project has already been completed in PR #3749. Building on top of this implementation provides the most practical path forward for tackling the problem. The overall implementation can be divided into four stages.

1. Rebase and Stabilise : The first step is to rebase this PR onto the current `master` branch of HLS. Since the PR was written against an older version of the codebase, significant changes have occurred in the interim, and rebasing will require resolving conflicts across several core files. While solving these conflicts we also have to make sure that the surrounding architecture remains correct.

After rebasing, the implementation must be verified to compile and run correctly. This includes ensuring that the existing test suite passes and confirming the expected runtime behaviour.In addition, PR #3749 depends on a fork of HieDb (`wz1000/HieDb`) that introduces utilities such as `addSrcFile`, `FakeFile`, and `lookupPackage`. Stabilising these utilities is also expected in this stage. (D1.)

2. Type-Level Rule : A major limitation of PR #3749 is the runtime whitelist used to prevent certain rules from executing on dependency files. This approach is fragile as consequently, new rules introduced in the future could accidentally execute on dependency files without detection.

The proposed solution is to enforce the distinction between file types directly in the type system. This is achieved using a new `InputClass` kind, a `RuleInput` type family, and a phantom-typed `InputPath` wrapper around `NormalizedFilePath`. Each rule declares the class of inputs it accepts, and the type system ensures that only compatible file types can be passed to that rule. Below is a demo implementation of this typed system.

Under this design, compilation rules such as `TypeCheck` and `GenerateCore` operate only on project files, while rules intended to work across all modules accept a broader input class. If an incompatible file type is passed to a rule, the violation is caught at compile time rather than at runtime. (D2.)

```haskell
data InputClass
   = ProjectHaskellFiles
   | AllHaskellFiles
   | CabalFile
   | StackYaml
   | NoFile

type family RuleInput key :: InputClass

newtype InputPath (i :: InputClass) =
   InputPath { unInputPath :: NormalizedFilePath }
   deriving newtype (Eq, Hashable, NFData, Typeable, Show)

type IdeRule k i v =
   ( Shake.RuleInput k ~ i
   , Shake.RuleResult k ~ v
   , Shake.ShakeValue k
   , Show v
   , Typeable v
    )
```

Implementing this change requires updating rule definitions across the codebase. All rules in `RuleTypes.hs` must declare their input class, and functions in `Shake.hs` that currently accept `NormalizedFilePath` must be updated to operate on `InputPath`. Corresponding updates will also be required in `Rules.hs` and in all plugins that invoke these functions (D3.).

Rule	Project File	Dependency File
`GetModificationTime`	✓ runs	✓ runs
`GetFileContents`	✓ runs	✓ runs
`GetModSummary`	✓ runs	✓ runs
`GetLocatedImports`	✓ runs	× blocked
`GhcSession`	✓ runs	× blocked
`GhcSessionDeps`	✓ runs	× blocked
`TypeCheck`	✓ runs	× blocked
`GenerateCore`	✓ runs	× blocked
`GetHieAst`	✓ runs	↺ from HieDb only
`GetModIface`	✓ runs	✓ runs
`NonLocalCompletions`	✓ runs	× blocked

Table 1: A partial list of rules and where to enforce them

3. Improving User Experience : This section involves uplifting the overall user experience of using this feature. This mainly involves two things to handle.

1) Readonly Enforcement : Source files generated for dependencies must be protected from accidental modification, which will be enforced through multiple layers to ensure consistent behaviour across different editors. At the editor level, HLS will update the project's VS Code `settings.json` to include a `files.readonlyInclude` rule, marking dependency files as read-only and preventing edits. As an additional safeguard for non–VS Code environments, generated files will have their write permissions removed at the operating system level, while keeping parent directories writable to allow future file generation. Finally, at the HLS level, dependency files are marked as read-only within the Files-of-Interest system (as introduced in PR #3749), ensuring that any `textDocument/didChange` notifications are ignored and no rebuild is triggered. (D4.)

2) HIE File Generation and Indexing : Reliable dependency navigation depends on the availability of HIE files, which are not generated by default and are absent for boot libraries. To evaluate and improve this workflow, experimentation with `-fwrite-ide-info` enabled for dependencies via Cabal configuration, rebuilding packages, and indexing the resulting `.hie` files using both `hiedb` and the `haskell-language-server hiedb` subcommand is needed. This will allow inspection of database contents, validation of symbol lookup behavior, and identification of friction in usability and generation, locating HIE files. Based on these findings, improvements will focus on automating HIE generation and indexing within HLS, improving user guidance, and exploring distribution strategies for prebuilt HIE files for boot libraries. (D5.)

4. Boot Library HIE Files : Obtaining HIE files for boot libraries remains an open problem. This is separate and must be developed once delivery of the top three stage is completed. The primary proposal is to distribute prebuilt HIE files together with a corresponding

HieDb database as a separate downloadable artifact for each GHC version. Instead of bundling these files directly with the GHC distribution, they would be provided as an optional package with a predictable structure. This approach has several advantages. First, it avoids inflating the size of standard GHC installations by keeping HIE metadata separate from the compiler binaries. Second, it provides a cleaner user experience. Tooling such as `ghcup` could expose a command to download the corresponding HIE files for the installed GHC version.

An important question is whether HIE files can become stale for a given GHC release. Since boot libraries change only when GHC itself is updated, it is likely that HIE files remain valid for the entire lifetime of a specific GHC version. Under this assumption, new HIE files would only need to be generated when a new GHC version is released.This suggests a straightforward maintenance strategy: whenever a new GHC version is released, an automated process could build the compiler with HIE generation enabled for boot libraries. These artifacts could then be published alongside the release. (D6.)

Multiple HieDb Instances. If boot library HIE files are distributed separately, HLS would effectively operate with two databases: one generated locally for project dependencies and another containing prebuilt metadata for boot libraries. One possible approach is to treat the boot library database as a fallback. If a lookup fails in the local HieDb, HLS could query the boot library database before returning a failure.

3 Road-map

3.1 Availability

I am currently a second-year master's student and have three classes per week during the project period. Aside from these commitments, I do not have any other planned obligations that would interfere with my work on this project. I will be able to dedicate approximately 30–40 hours per week toward development

3.2 Project Deliverables

These are a set of features / goals that i wish to accomplish by the end of GSOC tenure :

D1. Rebase PR #3749 onto the current HLS `master` branch and ensure the implementation compiles and behaves correctly with the latest codebase.

D2. Teach HLS when and when not to trigger rules by re-implementing PR #4449.

D3. Ensure complete coverage of rules by properly categorising each rule based on where it is allowed to run (e.g., project files, dependency files, all files, etc.).

D4. Improve the usability of generating and indexing HIE files for dependencies through better documentation, prompts from HLS, and clearer Cabal config docs.

D5. Ensure proper implementation of Read-only behavior on dependency files

D6. Investigate approaches for supporting boot libraries, including packaging HIE files separately from GHC or providing tooling support (e.g., via `ghcup`) to obtain them easily.

3.3 Timeline

The project will span approximately 12 weeks (350 hours). The first half of the project will focus on stabilizing the existing prototype and implementing the rule constraint infrastructure, while the second half will focus on integration, usability improvements, and extended support for boot libraries.

Weeks 1 – 2 : Rebase PR #3749 onto the current HLS `master` branch and ensure the implementation compiles correctly.

Weeks 3 – 5 : Reimplement the type-level rule constraint mechanism proposed in PR #4449 on top of the rebased implementation.

Weeks 6 : Use the feature locally to verify correct behavior and determine the appropriate set of rules for each input class. Once validated, extract the type-level rule constraint infrastructure into a standalone PR. This will form the primary deliverable for the midterm evaluation.

Weeks 7–8 : Finalize the rule constraint system and address any issues discovered during testing. Work towards merging the standalone rule constraint PR and landing the rebased implementation of PR #3749.

Weeks 9–11 : Improve the usability of generating and indexing HIE files for dependencies through documentation updates and improved guidance from HLS. Investigate possible approaches for supporting boot libraries and evaluate feasible distribution strategies for boot library HIE files. If feasible, prototype integration and verify compatibility with the existing indexing pipeline.

Weeks 12 : Stabilization phase. Address remaining issues, refine documentation, and prepare the feature for upstream integration into HLS.